

RollNumber: _____

Thapar Institute of Engineering & Technology
Department of Computer Science and Engineering
MID SEMESTER EXAMINATION

B.E. (2nd Yr. COE/CSE/AI-ML)

17th March, 2026

Monday, Time-11:30 AM to 1:30 PM

Time: 2 Hours

Max Marks: 60 (Weightage: 30)

Course Code: UCS415

Course Name: Design and Analysis of Algorithms

Name of Faculty: Dr. Tarunpreet (C1), Dr. Kapil (C2, Q1),

Dr. Suresh (C3, C5), Dr. Subhrendu (C4), Dr. Jyoti (C6, Q2), Dr.

Abdul (Q3, L1), Dr. Manpreet (C7, X1), Dr. Shruti (P1)

Note: Attempt all Questions in sequence. Answer all sub-parts of each question at one place. Mention Page No. of your attempt at front page of your answer sheet. Assume missing data (if any).

No	Questions	MM	CO	BL
Q.1	(a) Determine the worst-case time complexity of the following recurrence relation: $T(n) = 2T(\sqrt{n}) + 2 \log n$	(5)	CO1	L4
	(b) Suppose you wish to store a data file containing 100,000 characters, where only seven distinct characters appear with the frequencies specified in Table 1. Construct the optimal Huffman coding tree using the Huffman algorithm. Based on the constructed tree, determine the variable-length code assigned to each character and calculate the compression ratio achieved by Huffman coding compared to fixed-length coding. Show all necessary calculations. Assume that each frequency is given in thousands, and the minimum number of bits are required to represent each character in fixed-length coding.	(10)	CO2	L2
	(c) Given the mountain array $A = \{3, 8, 14, 19, 25, 31, 35, 42, 38, 30, 21, 12, 5\}$, find the peak element using divide and conquer strategy . An element is considered to be a peak element if it is strictly greater than its adjacent elements. If there are multiple peak elements, return the index of any one of them. Demonstrate how the search range changes after each iteration until the peak element is found. Assume that the element before the first element and the element after the last element are negative infinity and array indexing starts from 0.	(5)	CO1	L3
Q.2	(a) A space research organization receives signals from n satellites arranged in a linear order, and their signal strengths are stored in an array <code>strength[]</code> . Due to atmospheric interference, exactly one satellite is transmitting a faulty signal with value X , which is significantly weaker than the others. It is assumed that all other satellites transmit signals of equal strength, denoted by Y . i) Design an algorithm to efficiently identify the faulty satellite using the divide-and-conquer approach in which the array is divided into three equal parts at each step. ii) Write the recurrence relation for the algorithm. Solve the recurrence relation using the Master Method and determine the overall time complexity of the algorithm.	(4+4)	CO1	L2, L4
	(b) Using Dynamic Programming (DP), write the recursive equation to find the length of the Longest Common Subsequence (LCS) of two strings X and Y of lengths m and n , respectively. Given 2 strings $X = \text{"ABCBDAB"}$ and $Y = \text{"BDCABA"}$, answer the following: i) Show the complete DP table for the given strings and determine the length of the LCS. What is the space complexity of your approach? ii) Using the DP table, perform a recursive traceback to generate all possible distinct LCS strings (if more than one exists). Illustrate the traceback sequence of recursive calls involved in obtaining these sequences. iii) Can a space-optimized version of the LCS algorithm be used to compute the length of the LCS? Justify your answer and state the space complexity of the optimized approach. Will the space-optimized approach still work if you need to find all distinct LCS	(2)	CO2	L3, L4
		(4)		
		(3)		
		(3)		

	strings?			
3	(a) Consider the following pseudocode that prints sets of characters. In this program, the number of characters to be printed in the k^{th} set is specified by the array $call[k]$. The k^{th} invocation of the function $PS(k)$ is initiated after $delay[k]$ units of time. The function $recP(k)$ prints characters recursively, and each recursive call consumes exactly one unit of time. The time taken by the call to $PS()$ function and the printing of '[' and ']' is considered negligible. Since the program operates using a single function call stack, two executions of $PS()$ cannot overlap in time; otherwise, conflicts will occur due to simultaneous recursive activations. Therefore, it is required to design an elimination function that removes conflicting calls in such a way that the maximum possible number of printable sets can be executed without any overlap. In the function $eliminate(delay, call, n)$, the arrays $delay$ and $call$ are passed by reference, while n is passed by value. If an instance finishes execution at time t, the next instance may start immediately at time t, provided that the two executions do not overlap. <pre> recP(k) // recursively print { if (k == 0) return; print character ('a' + k); recP(k - 1); } PS(k) // print set { print '['; recP(k); print ']'; } main() { delay[] = {0, 1, 4, 7, 3}; call[] = {2, 4, 3, 2, 3}; n = 5; // size of delay n = eliminate(delay, call, n) // Body of for Loop is executing in parallel for different values of i for i= 0 to n-1 { while(delay[i] > 0) delay[i]--; PS(call[i]); } } </pre> i) Write the pseudo code of the function $eliminate(delay, call, n)$ using greedy algorithm that removes conflicting recursive calls to $PS()$ such that no two executions overlap in time and the maximum number of printable sets is retained. ii) Show step-by-step execution of eliminate function and write the output of the program after eliminations.	(5+5)	CO2	L3
	(b) A high-performance computing server executes n independent computational jobs $(J_1, J_2, \dots, J_n)$. During execution, each job produces an intermediate result that can optionally be stored in a limited-size cache to avoid recomputation in future executions. The system has a fixed cache capacity of C MB. The intermediate results generated by job J_i requires S_i MB of cache space. If the result of job J_i is cached, the system can save T_i milliseconds (ms) of recomputation time in future runs. Assume that partial caching of results is not allowed. Answer the following: i) Using dynamic programming, design the algorithms to compute the maximum recomputation time saved and determine the subset of jobs whose intermediate results should be cached, while ensuring that the total cache usage does not exceed the cache capacity. ii) Provide a step-by-step analysis of the worst-case time complexity of your algorithms.	(7+3)	CO2	L2, L3